

Mário

Etapa 1

Temos que pensar na lógica do nosso jogo!

Para isso, o que vocês podem perceber que acontece no Jogo?

1. O Mário Pula
2. O cenário mexe
3. Avisa o gameOver da partida
4. Reinicia o jogo se o usuário quiser

A partir dessa lógica, vamos construir nosso jogo!

Criando o cenário e os personagens do meu jogo

Para fazermos o Mário pular, antes de tudo, precisamos que o Mário e o obstáculo existam no nosso HTML

Dessa forma, é preciso criar a tela, o cenário e os nossos personagens no HTML! E você consegue pensar em quais tags HTML vamos utilizar?

OBS: peguem as imagens no repositório indicado pela professora!

Vamos iniciar criando nossos personagens:

```


```

E agora, analisando o nosso código, os personagens estão dentro de algum lugar?

Do meu jogo, certo?

E vocês conseguem pensar em alguma forma de manter os personagens dentro do meu jogo, uma tag HTML que permite que a gente guarde nossos elementos dentro de algum lugar?

```
<div class="jogo">
  

    
</div>
```

Agora que já criamos nossos personagens no HTML, precisamos estilizar com o CSS.

Para isso, compare o seu código com o jogo pronto e tente identificar as estilizações aplicadas ao personagem:

- Altura da imagem?
- Largura da imagem?
- A imagem tem borda?

Tanto para o obstáculo quanto para o Mário!

Em seguida, vamos analisar a estilização do cenário

- Ele possui um tamanho menor que o da tela?
- E a altura, como que ela é?
- Ele tem uma imagem como plano de fundo?
- Ou é uma cor?
- Ele tem borda?
- E o posicionamento em relação ao pai dele? Ele está no centro da página?

Para que a gente consiga alinhar no centro, utilizamos as propriedades `display`, `justify-content` e `align-items`!

Mas uma coisa muito importante para se fazer antes é adicionar mais um pai para os nossos elementos.

Com isso, definimos melhor as responsabilidades do nosso código. A <div> "jogo" fica responsável por falar o espaço que o cenário vai ocupar, além de centralizar o elemento na nossa página.

Enquanto a <div> "cenário" tem a responsabilidade de criar efetivamente o cenário (colocar a imagem, animação....)

```
.jogo {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  width: 100%;  
  height: 100%;  
}  
  
.cenario {  
  width: 1080px;  
  height: 380px;  
  border: 2px solid black;  
  background-image: url("cenario.jpg");  
}
```

Agora, só aplicarmos o posicionamento ao pai ao invés do filho!

Você sabe a diferença entre display flex e display grid? Ou porque utilizamos justify content e align-items, existem outras combinações?

Impedir meus personagens de fugirem do cenário

Como vocês podem observar, tanto o Mário quanto o obstáculo estão "fugindo" do nosso cenário.

Para que a gente consiga fixar eles dentro da tela do nosso jogo, precisamos aplicar a propriedade "position".

Utilizamos essa propriedade para conseguir colocar nossos elementos em qualquer parte da nossa página. É quase como um desenho: "Eu quero que fique mais na direita, no chão, no topo....."

Como padrão, todas as tags HTML possuem como posicionamento o "static". Ou seja, não é possível aplicar as propriedades **left**, **bottom**, **top** e **right**. Mas **O que elas significam?**

A propriedade **"bottom"** descreve o quão afastado o meu personagem está no chão. Enquanto o **"left"**, o quanto que está afastado da esquerda.

E, para **"right"**, a mesma lógica de antes para considerando a borda direita da nossa tela. E o **"top"**, você consegue adivinhar o que pode ser?

Para que os meus personagens saibam onde que eles devem ficar, eu preciso aplicar a seguinte propriedade ao meu pai:

```
.cenario {  
  width: 1080px;  
  height: 380px;  
  position: relative;  
  border: 2px solid black;  
  background-image: url("cenario.jpg");  
}
```

Dessa forma, eu falo para o meu cenário: "Ei, continua como você estava no seu fluxo original. Mas agora os seus filhos vão saber onde que você está posicionado!"

“Se você quiser se mexer mais para a esquerda ou mais para a direita em relação a tela do navegador, agora pode também!”

Em seguida, preciso avisar os meus filhos que o cenário vai passar a ser a referência de posicionamento deles, ao invés da página completa.

```
.mario {  
  height: 200px;  
  width: 120px;  
  position: absolute;  
  bottom: 0px;  
}
```

```
.obstaculo {  
  height: 160px;  
  width: 100px;  
  position: absolute;  
  bottom: 0px;  
}
```

O “position: absolute” remove por completo os meus personagens do fluxo original da página.

Como o pai possui “position: relative”, eles passam a utilizá-lo como ponto de referência.

Com isso, os personagens se mantêm dentro do cenário!

Pronto! Finalizamos nosso código.

Estilização do cenário:

```
.cenario {  
  width: 1080px;  
  height: 380px;  
  position: relative;  
  border: 2px solid black;  
  background-image: url("cenario.jpg");  
}
```

Fazer o cenário se movimentar

Para fazer o cenário se movimentar, precisamos aplicar propriedades de animação!

Mas, antes de tudo, vamos entender um pouco sobre as propriedades "transform" e "transition"! Com elas, podemos aplicar estilizações com CSS, mas de uma forma mais interativa (sem depender do Javascript!)

| **transform**: define **o que** vai ser alterado.

Podemos aumentar de tamanho, mover de um lado para o outro, girar, alterar a cor...

| **transition**: define **como** a alteração (transform) vai acontecer.

Ela vai se repetir várias vezes?

Por quanto tempo?

Vai ter o mesmo ritmo durante todo o tempo, ou ela vai começar devagar e terminar rápido?

Só que o "transition" precisa de um gatilho para ocorrer: quando o mouse passar em cima, quando o botão for clicado....

Mas o nosso cenário não se comporta assim. Ele não precisa esperar alguém clicar em um botão ou ter qualquer interação com a página, ela acontece sem parar!

Por isso, vamos utilizar a propriedade **animation**!

Complete o seu código com os comandos abaixo, antes da gente conversar um pouco sobre o que está acontecendo:

```
.cenario {  
  width: 1080px;  
  height: 380px;  
  overflow: hidden;  
  position: absolute;  
  border: 2px solid black;  
  background-image: url("cenario.jpg");  
  animation: moverCenario 2s linear infinite;  
}  
  
@keyframes moverCenario {  
  0% {  
    background-position: 0px;  
  }  
  100% {  
    background-position: -2000px;  
  }  
}
```

Quando utilizamos **@keyframes** podemos aplicar diversas alterações em um curto período de tempo ao nosso código, fornecendo uma movimentação e deixando a nossa página bem mais interativa!

Como o nosso cenário é um plano de fundo, vamos trabalhar com as propriedades que movem esse fundo!

Não se esqueça: eu só consigo animar ou alterar propriedades que não são absolutas, que podem se alterar! E, na maioria das vezes, vamos mexer com a largura e até mesmo o posicionamento dos elementos!

```
animation: moverCenario 2s linear infinite;
```

No código acima, eu chamo a animação que acabamos de criar e dou os seguintes comandos:

1. Faça por 2 segundos
2. Tenha o mesmo ritmo em toda animação (**linear**)
3. Repita infinitamente (**infinite**)

Mas o que vai durar pra sempre e por 2 segundos?

A animação que definimos!

```
@keyframes moverCenario {  
  0% {  
    background-position: 0px;  
  }  
  100% {  
    background-position: -2000px;  
  }  
}
```

Quando a animação começar (0%), a imagem **vai estar** no início do espaço que ela ocupa.

Lembre-se: quem definiu o espaço do cenário foi a nossa div “Jogo”! Então, a imagem **vai estar** posicionada exatamente no início dessa div!

Quando a animação estiver em 100% (ou seja, finalizou), a minha imagem vai se mover para fora do espaço determinado pelo elemento pai.

Só pensarmos em uma reta numérica: os números **negativos** estão do lado **esquerdo**, logo, a imagem também vai para o lado esquerdo, mas para fora da largura total que o pai dela possui

Com as características definidas com a propriedade "animation", o efeito do cenário passando infinitamente acontece!

Fazer o Mário pular

Como já sabemos como as animações funcionam, já podemos utilizar elas em outras partes do nosso código!

Nesse caso, para fazer o Mário pular!

Afinal de contas, quando o Mário pula, ele vai para onde?

- Vai para o topo e volta para o chão!

Como vamos definir o quanto que o Mário vai se afastar do chão, trabalhamos com a propriedade "bottom", em conjunto com as animações.

Fique a vontade para alterar a qual altura o Mário vai pular no "bottom"

```
@keyframes jump {  
  0% {  
    bottom: 0px;  
  }  
  50% {  
    bottom: 180px;  
  }  
  100% {  
    bottom: 0px;  
  }  
}
```

```
}  
}
```

No código acima, no início da animação a posição do Mário é `bottom = 0` (ou seja, não se afasta nada do chão). Na metade da animação, o Mário vai se afastar 180px e, ao final, o personagem retorna para o chão!

Igual a gente pula normalmente mesmo!

■ Não se esqueça de adicionar a animação no personagem que vai pular

```
.mario {  
  position: absolute;  
  height: 200px;  
  width: 120px;  
  bottom: 0px;  
  left: 100px;  
  animation: jump 0.5s linear infinite;  
}
```

Fazer o obstáculo se movimentar

Vamos utilizar a mesma lógica do Mário para adicionar o efeito de movimentação no nosso obstáculo!

Tudo o que precisamos fazer é dizer para ele: preciso que você ultrapasse a caixa do meu jogo infinitamente!

Então, vamos trabalhar com qual direção?

- Com a esquerda (left)!

E quantos segundos que essa animação vai durar?

- Vamos testar com 5 segundos!

Ela vai ser infinita?

- Com certeza!

A partir dessas respostas, vamos criar nossa animação e vê se ela vai dá certo!

```
@keyframes movimentacao {  
  0% {  
    left: 1100px;  
  }  
  50% {  
    left: 500px;  
  }  
  80% {  
    left: 50px;  
  }  
  100% {  
    left: -100px;  
  }  
}
```

```
.obstaculo {  
  height: 100px;  
  width: 100px;  
  position: absolute;  
  animation: movimentacao 2s linear infinite;  
  bottom: 0px;  
}
```

“Quando você tiver em determinado segundo, eu preciso que você tenha andado esse valor para a esquerda.”

Por fim, temos que adicionar na nossa div “cenario” o **overflow**, para que o pai esconda tudo o que ultrapassa a largura dele!

```
.cenario {  
  width: 1000px;  
  height: 380px;  
  overflow: hidden;  
  position: absolute;  
  border: 2px solid black;  
}
```

Vamos testar para vê se deu certo!

Avisa o gameover da partida

Pronto! Agora, vamos fazer a mágica acontecer.

Afinal de contas, eu preciso saber quando meu Mário vai pular (não quero que ele fique igual pipoca na panela infinitamente!) e quando que ele vai perder o jogo: ou seja, encostar no obstáculo!

- **Primeiro, precisamos saber se o Mário encostou**

Para descobrimos, precisamos verificar quando que o obstáculo vai está na mesma posição que o Mário (já que ele se mantém fixo!)

Então eu sei que quando o obstáculo estiver na mesma posição que o Mário, tem chances do encontro ter ocorrido. Mas qual posição que o Mário está? Longe da esquerda? Longe da direita?

Vamos olhar no css!

```
.mario {  
  position: absolute;  
  height: 100px;  
  width: 100px;  
  bottom: 0px;  
  left: 500px; //ou seja, se afasta da esquerda!
```

Como sabemos que o "left" (posição à esquerda) do Mário é 500, precisamos validar se o "left" do obstáculo está perto desse valor!

Como estamos trabalhando com animação, não é muito recomendado utilizar um valor absoluto. Pois os números se alteram muito rápido.

Dessa forma, vamos trabalhar com um intervalo de valores!

Antes de tudo, vamos seguir as etapas abaixo:

1. Chamar os personagens do HTML

```
let mario = document.querySelector(".mario");  
let obstaculo = document.querySelector(".obstaculo");
```

2. Para que a verificação ocorra somente quando o Mário encostar, vou guardar toda a minha lógica dentro de uma função. Defina o nome que você quiser pra ela.

3. Depois, precisamos saber o posicionamento do Mário, ou seja, o valor da propriedade "left"!

```
function marioEnconstou() {  
  const leftObstaculo = window.getComputedStyle(obstaculo).  
  left;  
  const valorConvertidoParaNumero = parseInt(leftObstaculo);  
  
  if (valorConvertidoParaNumero >= 500 && valorConvertidoParaNumero <= 510) {  
    return true;  
  }  
}
```

O `getComputedStyle` permite que a gente acesse o valor exato da propriedade! Graças às animações, esses valores se alteram constantemente!

Mas será que só isso é o suficiente?

Eu sei que o obstáculo está perto do personagem mas, mesmo se o obstáculo estiver na mesma posição, o Mário também pode pular!

- **Agora, vamos verificar se o Mário conseguiu pular!**

Se o Mário pular um valor maior que a altura do obstáculo, então ele não perdeu!

E qual é a altura do obstáculo no nosso código? Porque se o pulo for menor que esse valor, o Mário com certeza perdeu!

┃ Dica: procure no CSS!

Como o "getComputedStyle" retorna um texto, precisamos converter para um valor inteiro!

```
function marioNaoPulou() {  
    const bottomMario = window.getComputedStyle(mario).bottom;  
    m.replace("px", "");  
    const valorConvertidoParaNumero = parseInt(bottomMario);  
  
    if (valorConvertidoParaNumero < 70) {  
        return true;  
    }  
}
```

┃ Vamos vê como que ficou?

Agora, precisamos que toda essa verificação seja constante, para isso, vamos utilizar o setInterval.

```
setInterval(funcao, 1000);
```

Com esse comando, eu posso repetir o código a cada 10 milissegundos!

```
function game() {  
    loop = setInterval(() => {  
        if (marioEnconstou() && marioNaoPulou()) {  
            alert("Perdeu");  
        }  
    })  
}
```

```
    }, 10);  
  
}
```

- **Por fim, vamos avisar que o jogador perdeu!**

Para finalizar o meu jogo eu preciso:

- Que meu loop pare
- Meus personagens fiquem estáticos
- A caixinha para reiniciar o jogo apareça

1. Faço o loop parar

```
function GameOver() {  
    clearInterval(loop);  
}
```

2. Removo a animação dos personagens

Para isso, vamos ter que retornar ao css para remover a animação que ele aplicou!

```
.gameOver {  
  
    animation: none;  
  
}
```



```
function GameOver() {  
    clearInterval(loop);  
  
    obstaculo.classList.add("gameOver");  
    mario.classList.add("gameOver");  
  
}
```

3. Aviso o final da partida!

Quando vocês olham para a caixinha no projeto pronto, o que vocês acham que ela é, pensando em elementos HTML?

Aqui, vamos repetir o mesmo processo do formulário: criar elemento HTML utilizando o Javascript!

```
function GameOver() {  
    clearInterval(loop);  
  
    obstaculo.classList.add("gameOver");  
    mario.classList.add("gameOver");  
  
    const div = document.createElement("div");  
  
    div.innerHTML = "Fim da partida";  
}
```

Mas, eu preciso de um botão para que o meu jogo reinicie.

Por isso, ao invés de só adicionar um texto, eu vou adicionar essa tag HTML!

```
function GameOver() {
  clearInterval(loop);

  obstaculo.classList.add("gameOver");
  mario.classList.add("gameOver");

  const div = document.createElement("div");
  div.innerHTML = `
    <h2> Game over! </h2>
    <button onclick = "ReiniciarJogo()"> Jogar novamente </
button>

  `;
  reiniciar_jogo.appendChild(div);
}
```

Vamos tentar entender onde cada parte do código está.

- O "loop" existe no Javascript?
- O obstáculo ou o Mário existem no Javascript?
- E o nome gameOver? Representa o que?

Por fim, vamos identificar quais elemento são HTML e quais elementos são CSS!

A partir disso, a gente consegue conferir o que está faltando no nosso código!

Adicionar a classe "jump"

Agora, precisamos adicionar a lógica para o meu navegador saber quando que o Mário deve pular!

Além de adicionar a classe que vai fornecer o pulo para o Mário, precisamos, também, que ela reinicie!

Por isso, colocamos um `setTimeout` pra ele remover a classe logo depois que ela for adicionada (e 500 milissegundos nada mais é do que o tempo exato da minha animação!)

```
function jump() {  
  mario.classList.add("jump");  
  
  setTimeout(() => {  
    mario.classList.remove("jump");  
  }, 500);  
}
```

```
document.addEventListener("keydown", jump);  
game();
```

O evento é necessário para que o Javascript escute sempre que a gente selecionar uma tecla do teclado.

Utilizamos o `setTimeout` ao invés do `SetInterval` pois ele vai executar a função uma única vez. Sempre que alguém clicar em alguma tecla do teclado, a função adiciona a classe "jump", espera a animação acabar e remove a classe logo em seguida!

```
.jump {  
  animation: jump 0.5s linear;  
}
```

Reiniciar o jogo

E agora, o que está faltando?

Para que seja possível reiniciar o jogo, precisamos:

1. Limpar todo o conteúdo da div que mostra o gameover.
2. Depois, retomar a animação dos dois elementos (Mário e obstáculo).
3. Por fim, chamar a função `game()`, para que o jogo efetivamente reinicie.

Adicione por conta própria esse final e veja o que acontece!